

HW4: VDN on MPE simple_tag

PKU Reinforcement Learning Course

MAK CHAK WING UID: 2501213361

2026

Contents

1	Environment & Problem Setting	2
2	VDN Method	2
2.1	Additive Value Decomposition	2
2.2	IGM Property	2
2.3	Per-Agent Q-Network and Training Stabilisers	2
3	Implementation Details & Hyperparameters	3
4	Training Results	4
5	Evaluation Results	5
6	Discussion	6
7	Limitations & Future Work	7
8	Conclusion	7
9	How to Run	8
9.1	Prerequisites	8
9.2	Install Dependencies	8
9.3	Train (Full Pipeline)	8
9.4	Evaluate Best Checkpoint	8
9.5	Record Evaluation Video	8
9.6	Quick Sanity Check	8

1 Environment & Problem Setting

simple_tag_v3 (PettingZoo MPE) is a cooperative predator-prey task on a 2-D continuous plane. Three adversary agents (predators) must cooperate to catch one prey agent while navigating around two circular obstacles. Episodes last 100 steps; only the adversaries are trained.

Observation space (adversary, 16-D continuous): self velocity (2-D), self position (2-D), relative positions of 2 obstacles (4-D), relative positions of 2 other adversaries (4-D), relative position and velocity of the prey (4-D). Each adversary observation is augmented with a 3-D one-hot agent ID before the network, giving an effective input of 19-D.

Action space: 5 discrete actions per adversary: {no-op, left, right, down, up}.

Reward structure. A collision between any adversary and the prey adds +10 to all three adversaries simultaneously (and −10 to the prey). All other rewards are zero. This shared, team-level bonus motivates CTDE: optimising each agent independently can encourage one agent to do most of the work while the others stagnate (a “lazy-agent” failure mode), so a joint value function is used to align individual incentives with the team objective.

2 VDN Method

2.1 Additive Value Decomposition

VDN decomposes the joint action-value as a sum of per-agent utilities:

$$Q_{\text{tot}}(o, a) = \sum_{i=1}^N Q_i(o_i, a_i)$$

where $N = 3$, o_i is agent i ’s local observation, and a_i is its discrete action. Each Q_i depends only on agent i ’s own observation–action pair, so execution requires no inter-agent communication.

2.2 IGM Property

Because Q_{tot} is a sum of independent terms, joint greedy action selection decomposes into per-agent argmaxes:

$$\arg \max_a Q_{\text{tot}}(o, a) = \left(\arg \max_{a_1} Q_1, \dots, \arg \max_{a_N} Q_N \right)$$

This Individual-Global-Max (IGM) property makes execution fully decentralised: each agent acts greedily on its own Q_i without coordinating with teammates, while training still optimises the team objective through the joint TD target.

2.3 Per-Agent Q-Network and Training Stabilisers

A **single shared-parameter network** Q_θ is used for all three adversaries (EPyMARL “S” variant). The input is $[o_i; \text{id}_i]$ where $\text{id}_i \in \{0, 1\}^3$ is a one-hot agent identifier intended to help break weight-sharing symmetry so agents can specialise into distinct roles. Architecture: 2-layer MLP ($19 \rightarrow 128 \xrightarrow{\text{ReLU}} 128 \xrightarrow{\text{ReLU}}$) with a dueling head that splits the representation into a scalar value $V(h)$ and a mean-centred advantage stream $A(h, a)$, recombined as $Q = V + A - \bar{A}$.

Double-DQN target. Action selection uses the online network θ ; value estimation uses the target network θ^- , reducing positive bias in the joint TD target: $y = r + \gamma(1-d) \sum_i Q_{\theta^-}(o'_i, \arg \max_{a'} Q_{\theta}(o'_i, a'))$.

Huber loss. Smooth-L1 loss bounds the gradient magnitude for large early-training TD errors, preventing instability from a randomly initialised Q-network.

Hard target update. θ^- is hard-copied from θ every 500 gradient steps, providing a stable regression target between copies.

3 Implementation Details & Hyperparameters

Joint replay buffer. A single buffer stores joint transitions $(O_t, a_t, r_t, O_{t+1}, d_t)$ with O_t shaped $(3, 19)$. Each sampled batch of size 128 yields $128 \times 3 = 384$ agent-level transitions simultaneously.

Training vs. evaluation reward. Training uses a shaped reward $r_{\text{train}} = r_{\text{col}}/10 - 0.1 \sum_i \|\text{prey_rel}_i\|$ that adds a dense distance-penalty to the scaled collision bonus. Evaluation uses only the raw collision sum (+10 per hit), with no shaping, to measure true task performance.

Best-checkpoint saving. Whenever the 30-episode greedy mean improves, weights are saved to `best_model.pt`. This is essential: as shown below, the final-episode weights are 11% worse than the peak.

Table 1: Hyperparameters used for VDN on simple_tag_v3.

Parameter	Value
Environment	MPE simple_tag_v3
Algorithm	VDN
Adversaries / prey / obstacles	3 / 1 / 2
Episodes / steps per episode	100,000 / 100
Network hidden units / layers	128 / 2
Dueling head / Double DQN	Yes / Yes
Parameter sharing + one-hot agent ID	Yes
Discount factor γ	0.99
Learning rate	10^{-4} (Adam)
Batch size	128
Replay buffer size	100,000
Gradient clip (ℓ_2 norm)	10.0
Target update interval	500 gradient steps (hard)
Learn every / Warmup steps	32 env steps / 5,000
ε : 1.0 $\rightarrow$ 0.05 over	4,000,000 env steps
Reward scale / distance shaping	10.0 / 0.1
Loss	Smooth-L1 (Huber)
Prey policy / Seed	Fixed random / 42

4 Training Results

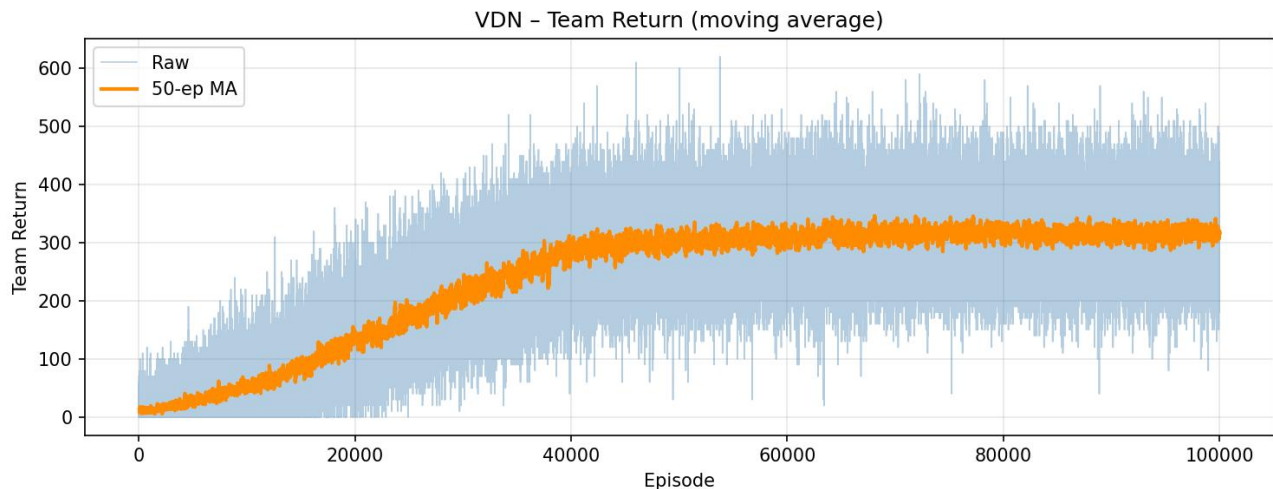


Figure 1: Team return per episode (blue) and 50-episode moving average (orange) over 100,000 training episodes (10,000,000 environment steps).

The adversaries learn cooperative pursuit from scratch over 100,000 episodes. The raw episode return (blue) is highly variable throughout due to stochastic ε -exploration and the binary collision reward structure; the 50-episode moving average (orange) provides a cleaner trend.

Early phase (episodes 0–10,000). Training begins with near-random exploration (ε above 0.7). The team return rises slowly from near-zero as agents learn basic movement toward the prey. The distance-shaping term provides the only gradient signal during this phase: without it, the collision reward is too sparse (fewer than 2 events per episode) to drive meaningful learning.

Rapid improvement (episodes 10,000–35,500). As ε decays below 0.5 and the buffer accumulates more on-policy data, the moving average rises steeply. The reward trend in this phase is consistent with improving cooperative pursuit: the 50-ep moving average peaks at **345.67** at episode **35,500**, corresponding to roughly 34–35 collision events per 100-step episode. The recorded demo video shows the three adversaries approaching the prey from different directions, although we did not run an ablation to confirm that this is what the reward gain measures.

Plateau and regression (episodes 35,500–100,000). Once ε floors at 0.05 (step 4,000,000, episode 40,000), the policy is nearly greedy. Continued gradient updates on a near-fixed behaviour distribution push Q-values off the optimal surface without fresh exploration to correct them. The moving average regresses from 345.67 to a final eval of **307.33**, an 11% drop from peak, confirming the importance of best-checkpoint saving.

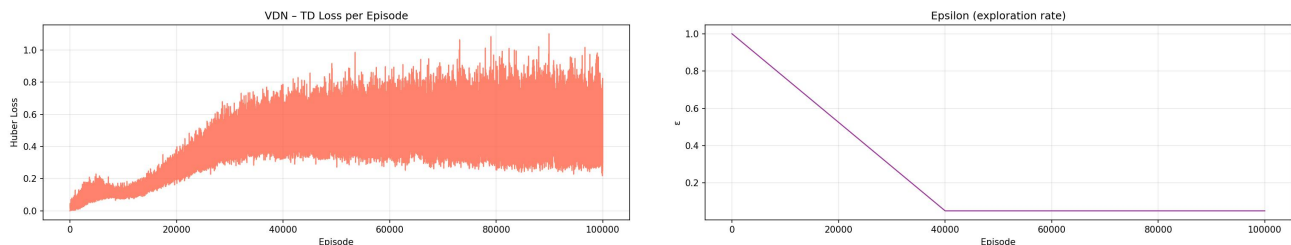


Figure 2: Left: TD loss (peaks during rapid improvement, stabilises as Q-values converge). Right: ε decay schedule, linear from 1.0 to 0.05 over 4,000,000 env steps, flooring at episode 40,000.

5 Evaluation Results

Every 500 training episodes, the current network is evaluated over **30 greedy episodes** ($\arg \max_a Q_i(o_i, a)$, no ε -noise) using the raw collision sum only.

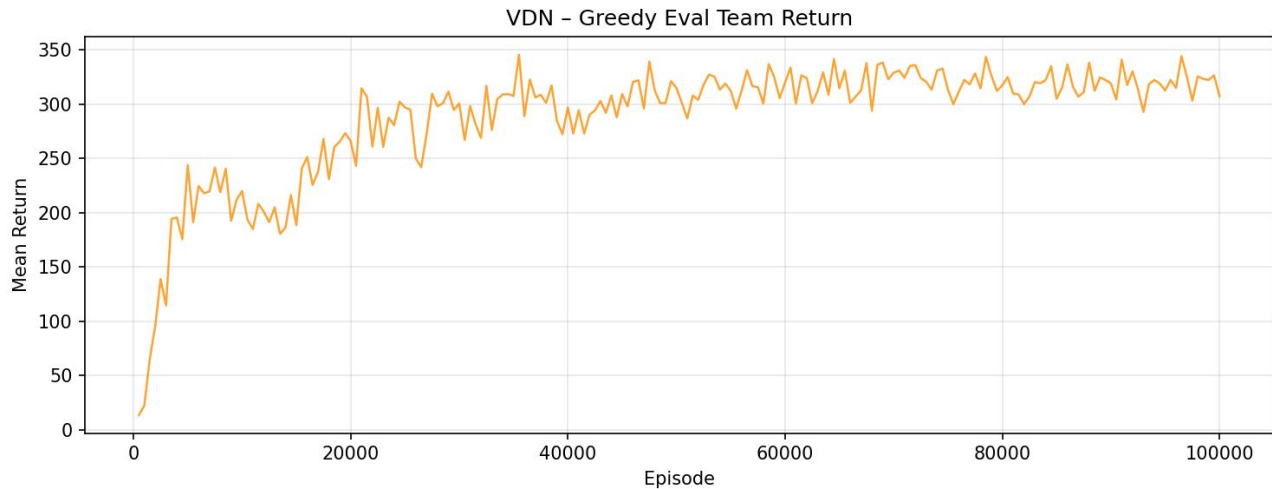


Figure 3: Greedy evaluation return (mean over 30 episodes) measured every 500 training episodes. Best: 345.67 at episode 35,500.

The eval return rises from **13.67** at episode 500 to a peak of **345.67** at episode **35,500**. After ε hits its floor at episode 40,000, the eval settles in the 297–329 band with occasional dips. Key milestones are listed below.

Table 2: Key training milestones.

Milestone	Value
Best greedy eval (mean over 30 ep)	345.67 at episode 35,500
Final eval (episode 100,000)	307.33 ± 58.76
$\varepsilon = 0.1$ reached	step 3,789,474 (episode 37,895)
$\varepsilon = 0.05$ reached	step 4,000,000 (episode 40,000)
Total env steps / wall-clock	10,000,000 / approx. 1 h 03 min (CPU)

The high standard deviation of the final eval (58.76) reflects the stochasticity of the fixed-seed prey trajectory: in some episodes the random prey wanders into an adversary cluster (many collisions), while in others it drifts to a corner far from the team (few collisions). The adversary policy itself is deterministic at evaluation; the variance comes entirely from the environment.

6 Discussion

Cooperative pursuit and parameter sharing. The reward trend suggests improving cooperative pursuit within approximately 15,000 episodes, reaching 30–35 collisions per 100-step episode at peak. One ingredient that may contribute to this is shared-parameter training with one-hot agent IDs. Without an identifier, all three adversaries would receive identical inputs under shared weights, which is a known failure pattern in shared-parameter MARL where agents collapse onto the same policy (the “lazy-agent” failure mode). Adding the one-hot ID is intended to help break this symmetry, so that despite sharing weights each agent conditions its Q-values on its own identifier and can in principle specialise into a distinct approach angle. We did not run an ablation removing the one-hot ID, so the contribution of this design choice is not directly measured here. The episode-length diagnostic (Figure 2) is consistent with this picture: average episode length drops from 100 steps (prey never caught) to around 55 steps by episode 15,000, suggesting that the team learns to reach the prey within the first half of each episode.

Late-stage plateau and regression. The learning curve peaks at episode 35,500 and then regresses by 11% to 307.33 at the final episode, a pattern familiar from single-agent DQN. With ε floored at 0.05, the behaviour distribution is nearly fixed, and each gradient step slightly overfits to the current replay buffer. Because the VDN target itself depends on θ (via the Double-DQN action-selection step), small weight changes compound over thousands of updates, slowly drifting Q-values away from the optimum. Double-DQN and gradient clipping both dampen this drift, but they do not eliminate it under a constant learning rate. The consequence is that the best checkpoint (episode 35,500) diverges meaningfully from the final checkpoint (episode 100,000): evaluating only the final weights would understate performance by 38 reward points. This makes **best-checkpoint saving non-negotiable** in this setup. A learning-rate decay schedule (cosine annealing to 10^{-5} after ε floors) would likely close this gap by reducing the magnitude of late-training updates.

Scripted flee failure. A scripted flee policy was tested as an alternative prey: the prey moves directly away from the nearest adversary at each step, producing deterministic evasion. VDN training under the scripted flee policy produces near-zero evaluation returns throughout the full 100,000-episode run; the agents never learn to catch the prey at all. The root cause is the feed-forward MLP’s inability to plan multi-step encirclement. Catching a deterministically evasive prey requires the adversaries to predict the prey’s future trajectory and coordinate an encirclement manoeuvre over 5–10 future steps. A flat MLP evaluated with single-step TD cannot represent such strategies: it can react to the current observation but cannot model the consequences of coordinated future actions. The random-prey experiment succeeds precisely because the random prey does not exploit the adversaries’ predictability: it moves randomly and occasionally steps toward an adversary, enabling collisions even with a greedy single-step policy.

Distance shaping as a dense curriculum. The shaping term $-0.1 \sum_i \|\text{prey_rel}_i\|$ provides a continuous gradient from episode one, guiding agents toward the prey even before any collision occurs. Without it, the collision reward arrives fewer than 2 times per episode for the first several thousand episodes, stalling learning entirely. Once the team reliably reaches the prey (around episode 10,000), the shaping becomes secondary relative to the growing collision signal. Shaping is disabled at evaluation so that the reported return reflects the true task objective.

7 Limitations & Future Work

The current implementation has several deliberate simplifications that point to clear improvement directions:

- **QMIX (monotonic mixing).** VDN’s additive decomposition is a special case of QMIX’s monotone mixing network. QMIX can represent a strictly larger class of cooperative value functions while preserving IGM via monotonicity. In environments where per-agent values cannot be cleanly separated (for instance when agents must jointly time their approach), QMIX’s non-linear mixing can capture the coordination structure that VDN cannot.
- **Recurrent backbone (GRU).** Replacing the MLP body with a gated recurrent unit would give each agent a belief state over partial observations across time, enabling the multi-step encirclement reasoning required against the scripted flee prey. The GRU hidden state is reset at episode boundaries and sampled from episode sequences in the replay buffer.
- **Prioritised experience replay.** Collision events are rare early in training ($< 2\%$ of steps), so the uniform buffer undersamples them relative to their information content. Prioritising high-TD-error transitions would upsample the rare collision signal and accelerate early learning without hand-crafted shaping.
- **Prey self-play.** Once a recurrent backbone is in place, replacing the fixed-random prey with a concurrently trained agent would drive the adversaries toward more robust pursuit strategies that generalise beyond the random-walk opponent. A curriculum starting from a weak prey and gradually increasing its capability would prevent divergence in early training.
- **Learning-rate decay.** A cosine-annealing schedule to 10^{-5} after ε floors would reduce the magnitude of late-training updates and bring the final-checkpoint and best-checkpoint returns closer together, reducing reliance on checkpoint selection.

8 Conclusion

This homework implements VDN on the MPE simple_tag_v3 cooperative predator-prey task. The key observations are:

- **Parameter sharing with one-hot agent IDs is intended to help break symmetry.** All three adversaries share one Q-network, and the one-hot ID is included so that each agent can in principle specialise into a distinct pursuit role despite shared weights. The team’s reward trend reaches 34–35 collisions per 100-step episode at peak; an ablation would be needed to attribute this directly to the agent ID.
- **Distance shaping is essential for sparse-reward cooperative tasks.** The dense shaping term provides gradient signal from episode one and is safely removed at evaluation without biasing the reported metric.
- **Best-checkpoint saving is non-negotiable under a constant learning rate.** Peak performance (345.67 at episode 35,500) is 11% better than the final-episode checkpoint (307.33 ± 58.76). The best-checkpoint mechanism preserves the peak (345.67 from `best_model.pt`) rather than the regressed final-episode weights.
- **A feed-forward MLP with single-step TD cannot catch a deterministically evasive prey.** The scripted flee experiment confirms that recurrent memory or look-ahead planning is required.

9 How to Run

9.1 Prerequisites

- Python 3.8+

9.2 Install Dependencies

```
pip install torch numpy matplotlib pettingzoo[mpe] imageio imageio-ffmpeg tqdm
```

9.3 Train (Full Pipeline)

```
python Code.py --mode train --episodes 100000
```

Trains for 100,000 episodes (10,000,000 env steps), evaluates every 500 episodes with 30 greedy rollouts, saves the best checkpoint to `checkpoints/best_model.pt`, and writes plots to `plots/`.

9.4 Evaluate Best Checkpoint

```
python Code.py --mode eval \  
    --checkpoint checkpoints/best_model.pt
```

Loads `best_model.pt` and runs 30 greedy evaluation episodes, printing mean $\pm$ std return.

9.5 Record Evaluation Video

```
python Code.py --mode record \  
    --checkpoint checkpoints/best_model.pt \  
    --video-seconds 30
```

Records 30 seconds of greedy evaluation and saves to `videos/eval_demo.mp4` at 10 fps.

9.6 Quick Sanity Check

```
python Code.py --mode train --smoke
```

Runs a reduced smoke-test (25 episodes, minimal buffer) to verify the environment and training loop work correctly.